

LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III



Acara 14

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

Bab 1. Pendahuluan

1.1 Latar Belakang

Aplikasi berbasis web tidak hanya dituntut mampu menerima, mengolah, dan menyimpan data, tetapi juga harus mampu menjaga keamanan data serta menangani kesalahan yang mungkin terjadi selama proses aplikasi berjalan. Pada aplikasi Sistem Informasi Akademik, pengguna dapat melakukan berbagai aktivitas seperti menambahkan, menampilkan, mengubah, dan menghapus data mahasiswa. Data yang diberikan pengguna melalui form selanjutnya diproses oleh aplikasi dan disimpan ke dalam database MySQL.

Apabila input pengguna langsung diteruskan ke database tanpa pemeriksaan, aplikasi dapat menerima data yang tidak sesuai dengan ketentuan. Selain itu, pembuatan query SQL yang tidak aman dapat menyebabkan serangan SQL Injection. Oleh karena itu, diperlukan validasi input dan penggunaan prepared statement untuk memisahkan perintah SQL dari data yang diberikan pengguna.

Selain aspek keamanan, aplikasi juga perlu mampu menangani kesalahan secara baik. Kesalahan seperti kegagalan koneksi database atau kesalahan query seharusnya tidak menampilkan informasi teknis kepada pengguna karena informasi tersebut dapat membingungkan pengguna serta berpotensi membuka informasi mengenai struktur internal aplikasi. Exception Handling digunakan untuk menangkap kesalahan tersebut, sedangkan Logging digunakan untuk mencatat detail kesalahan sehingga dapat diperiksa oleh pengembang.

Aplikasi juga perlu memberikan informasi mengenai hasil operasi kepada pengguna melalui Flash Message. Setelah melakukan operasi yang mengubah data seperti Create, Update, atau Delete, aplikasi menggunakan pola Post/Redirect/Get atau PRG. Pola tersebut digunakan agar browser tidak mengirimkan ulang data POST ketika pengguna melakukan refresh halaman.

Pada praktikum Acara 14, seluruh konsep tersebut diterapkan pada CRUD data mahasiswa dalam Sistem Informasi Akademik. Aplikasi menggunakan arsitektur Controller, Service, Repository, Model, dan MySQL sehingga setiap bagian aplikasi mempunyai tanggung jawab yang jelas.

1.2 Tujuan Praktikum

Tujuan dari praktikum Acara 14 adalah:

1. Mengintegrasikan aplikasi CRUD Data Mahasiswa dengan database MySQL.
2. Memahami pentingnya keamanan pada aplikasi web.
3. Menerapkan validasi input sebelum data diteruskan ke database.

4. Menggunakan PDO Prepared Statement untuk membantu mencegah SQL Injection.
5. Menerapkan Service Layer untuk menangani validasi dan business logic.
6. Menerapkan Repository Pattern sebagai bagian yang bertanggung jawab terhadap akses database.
7. Menerapkan Exception Handling menggunakan mekanisme try-catch.
8. Membuat sistem Logging untuk mencatat detail kesalahan aplikasi.
9. Menyimpan log kesalahan ke dalam storage/logs/app.log.
10. Menampilkan pesan yang aman kepada pengguna menggunakan Flash Message.
11. Menerapkan pola Post/Redirect/Get untuk mencegah pengiriman form secara berulang ketika halaman di-refresh.
12. Mengimplementasikan operasi CRUD lengkap yang meliputi Create, Read, Update, dan Delete.
13. Memahami alur data mulai dari User, Controller, Service, Repository, hingga MySQL.
14. Memisahkan informasi teknis untuk programmer dengan pesan yang diberikan kepada pengguna.

Bab 2. Dasar Teori

2.1 Security pada Aplikasi Web

Security atau keamanan aplikasi web merupakan serangkaian mekanisme yang digunakan untuk melindungi aplikasi dan data dari input atau tindakan yang tidak diinginkan. Dalam aplikasi Sistem Informasi Akademik, data yang dimasukkan pengguna tidak dapat langsung dianggap benar sehingga perlu diperiksa terlebih dahulu sebelum diproses. Beberapa keamanan dasar yang diterapkan pada praktikum ini adalah validasi input, prepared statement, exception handling, dan penyembunyian detail kesalahan database dari pengguna.

2.2 Validasi Input

Validasi input merupakan proses memeriksa data yang diberikan pengguna sebelum data diproses lebih lanjut. Validasi dilakukan untuk memastikan bahwa data memenuhi aturan aplikasi.

Pada aplikasi Data Mahasiswa, beberapa validasi yang digunakan antara lain:

- NIM tidak boleh kosong.
- NIM harus berupa angka.
- Nama mahasiswa wajib diisi.
- Email wajib memiliki format yang benar.
- Program studi harus dipilih.
- Tahun angkatan harus berada pada rentang yang diperbolehkan.
- Dua digit pertama NIM harus sesuai dengan dua digit terakhir tahun angkatan.
- Status hanya boleh berupa aktif, cuti, atau lulus.

Dengan menggunakan validasi, data yang tidak sesuai aturan dapat dihentikan sebelum dikirim ke Repository dan database.

2.3 PDO

PDO atau PHP Data Objects merupakan antarmuka PHP yang digunakan untuk berkomunikasi dengan database. Pada aplikasi Sistem Informasi Akademik, PDO digunakan untuk menghubungkan aplikasi PHP dengan MySQL.

2.4 Prepared Statement

Prepared Statement digunakan untuk menjalankan SQL dengan memisahkan perintah SQL dari data yang diberikan pengguna. Cara tersebut lebih aman dibandingkan menyusun query menggunakan penggabungan string dan membantu mencegah SQL Injection.

2.5 Repository Pattern

Repository merupakan lapisan yang bertugas melakukan akses terhadap database. Pada praktikum ini digunakan class MahasiswaRepository. Dengan adanya Repository, SQL tidak

dituliskan di Controller. Controller cukup meminta Service untuk menjalankan proses, kemudian Service menggunakan Repository untuk mengakses database.

2.6 Service Layer

Service merupakan lapisan yang menangani business logic aplikasi. Pada praktikum ini digunakan MahasiswaService. Dengan demikian Controller tidak perlu berisi validasi maupun SQL.

2.7 Exception Handling

Exception Handling merupakan mekanisme untuk menangani kesalahan pada aplikasi agar aplikasi tidak langsung berhenti dan menampilkan detail teknis kepada pengguna. Pada praktikum ini, apabila terjadi kesalahan database, detail exception diberikan kepada Logger sedangkan pengguna hanya menerima pesan sederhana seperti:

2.8 Logging

Logging adalah proses mencatat kejadian atau kesalahan yang terjadi pada aplikasi. Log diperlukan oleh programmer untuk mengetahui jenis kesalahan, waktu terjadinya kesalahan, file yang bermasalah, dan baris kode tempat terjadinya kesalahan. Pada praktikum ini log disimpan pada storage/logs/app.log.

2.9 Session dan Flash Message

Session digunakan untuk menyimpan data sementara pada server. Salah satu penggunaannya adalah untuk menyimpan Flash Message. Flash Message digunakan untuk memberikan feedback setelah proses dilakukan.

2.10 Post/Redirect/Get

Post/Redirect/Get atau PRG merupakan pola yang digunakan setelah form dikirim menggunakan method POST. Setelah redirect dilakukan, request terakhir browser adalah GET. Oleh karena itu, ketika pengguna melakukan refresh, browser hanya mengulangi GET dan tidak mengirim ulang form POST. Hal tersebut mencegah terjadinya duplikasi data.

2.11 CRUD

CRUD merupakan singkatan dari Create, Read, Update, dan Delete. Pada aplikasi Data Mahasiswa:

Create Menambahkan mahasiswa

Read Menampilkan mahasiswa

Update Mengubah mahasiswa

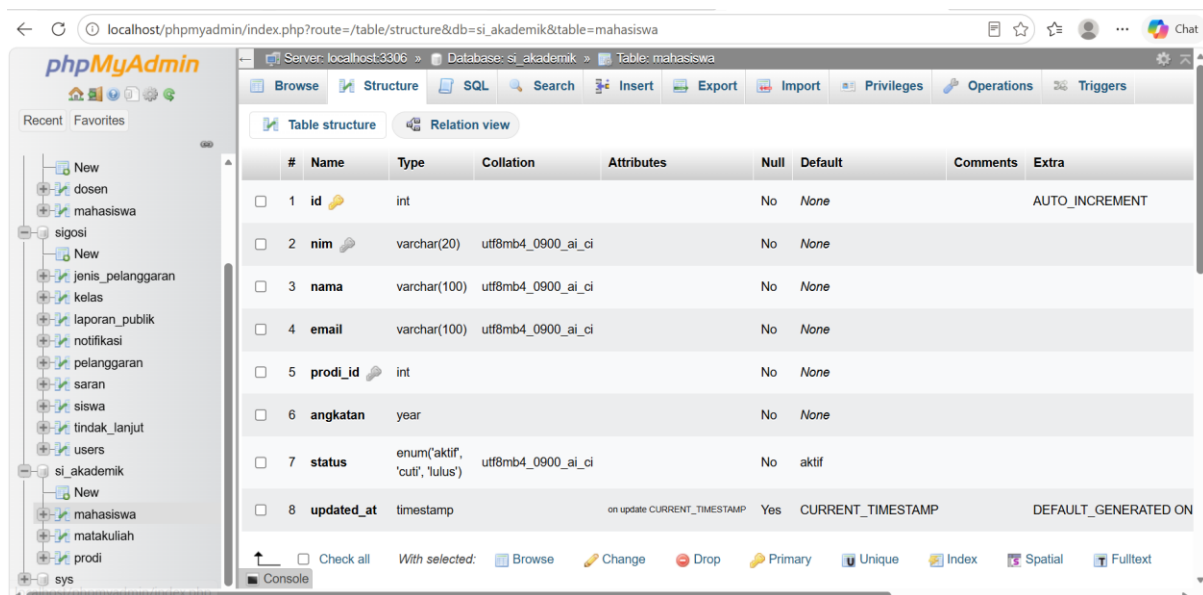
Delete Menghapus mahasiswa

Seluruh operasi tersebut terhubung dengan database MySQL menggunakan Repository.

Bab 3. Hasil Praktikum

3.1 Persiapan Database

Pada praktikum digunakan database si_akademik. Database yang digunakan telah memiliki tabel mahasiswa, prodi, dan matakuliah. Tabel mahasiswa memiliki field ID, NIM, nama, email, program studi, angkatan, status, dan waktu pembaruan data.



3.2 Koneksi Aplikasi dengan MySQL Menggunakan PDO

Class Database menggunakan pola Singleton sehingga object koneksi database tidak dibuat berulang kali.

PDO dikonfigurasi menggunakan mode exception `PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION` sehingga apabila terjadi kesalahan database, PDO menghasilkan PDOException yang selanjutnya dapat ditangani aplikasi.

```

51 // Isi log
52 $logMessage =
53     date('Y-m-d H:i:s') .
54     ' - Database Connection Error - ' .
55     $e->getMessage() .
56     ' - File: ' .
57     $e->getFile() .
58     ' - Line: ' .
59     $e->getLine() .
60     PHP_EOL;
61
62 // Simpan ke app.log
63 file_put_contents(
64     $logFile,
65     $logMessage,
66     FILE_APPEND
67 );
68
69 // Pesan aman untuk pengguna
70 die(
71     'Koneksi database gagal. Silakan coba lagi.'
72 );
73 }
74 }
75

```

3.3 Implementasi Repository dan Prepared Statement

```

4 references | 0 overrides
236 public function existsByNim(
237     string $nim,
238     ?int $excludeId = null
239 ): bool {
240
241     $sql = "
242         SELECT COUNT(*)
243         FROM mahasiswa
244         WHERE nim = :nim
245     ";
246
247     if ($excludeId !== null) {
248         $sql .= "
249             AND id != :id
250         ";
251     }
252
253     $stmt = $this->db->prepare($sql);
254
255     $params = [
256         'nim' => $nim
257     ];
258
259     if ($excludeId !== null) {
260         $params['id'] = $excludeId;

```

3.4 Implementasi MahasiswaService

Business logic dan validasi ditempatkan pada: app/Services/MahasiswaService.php. Service menerima MahasiswaRepository, ProdiRepository, dan LoggerService melalui constructor Dependency Injection. Penggunaan Service membuat Controller lebih sederhana karena validasi dan logika aplikasi tidak dituliskan langsung pada Controller.

```
19     public function __construct(  
20         Database $database  
21     ) {  
22         $this->db =  
23             $database->getConnection();  
24     }  
25  
26  
27     /**  
28      * Menampilkan data mahasiswa.  
29      */  
30     public function all(  
31         ?string $keyword = null,  
32         int $limit = 5,  
33         int $offset = 0  
34     ): array {  
35  
36         $sql = "  
37             SELECT  
38                 m.id,  
39                 m.nim,  
40                 m.nama,  
41                 m.email,  
42                 m.prodi_id,  
43                 m.angkatan,  
44                 m.status,
```

3.5 Implementasi Validasi Input

Validasi mahasiswa dilakukan di method validate(). Apabila input tidak memenuhi aturan maka aplikasi menghasilkan InvalidArgumentException. Pesan validasi kemudian dapat diberikan kepada pengguna menggunakan Flash Message.

```

67 ✓ public function setId(?int $id): void
68 {
69 ✓     if ($id !== null && $id < 1) {
70         throw new InvalidArgumentException(
71             'ID tidak valid.'
72         );
73     }
74
75     $this->id = $id;
76 }
77
78
20 references | 0 overrides
79 ✓ public function getNim(): string
80 {
81     return $this->nim;
82 }
83
84
12 references | 0 overrides
85 ✓ public function setNim(string $nim): void
86 {
87     $nim = trim($nim);
88
89 ✓     if ($nim === '') {
90         throw new InvalidArgumentException(

```

3.6 Implementasi Exception Handling

Operasi Create, Update, dan Delete ditempatkan di dalam blok try-catch. Detail kesalahan database tidak diberikan kepada pengguna. Pengguna hanya menerima informasi bahwa proses gagal.

```

1 |
2     } catch (PDOException $e) {
3
4         // Lokasi folder log
5         $logDirectory =
6             __DIR__ . '/../../storage/logs';
7
8         // Buat folder logs jika belum ada
9         if (!is_dir($logDirectory)) {
10             mkdir(
11                 $logDirectory,
12                 0777,
13                 true
14             );
15         }
16     }

```

3.7 Implementasi Logging

Logging dibuat menggunakan `app/Services/LoggerService.php`. Apabila terjadi exception, Logger mencatat detail kesalahan ke `storage/logs/app.log`. Data yang dicatat antara lain waktu, jenis exception, pesan exception, file, dan nomor baris. Dengan cara tersebut programmer tetap dapat mengetahui penyebab error tanpa menampilkan detail teknis tersebut kepada pengguna.

```
1  <?php
2
3  namespace App\Services;
4
5  use Throwable;
6
7  4 references | 0 implementations
8  class LoggerService
9  {
10     /**
11      * Menyimpan error ke storage/logs/app.log.
12      */
13     6 references | 0 overrides
14     public function error(Throwable $e): void
15     {
16         $logDirectory =
17             __DIR__ .
18             '/../..../storage/logs';
19
20         /**
21          * Jika folder belum ada,
22          * buat secara otomatis.
23          */
24         if (!is_dir($logDirectory)) {
25             mkdir(
26                 $logDirectory,
```

3.8 Implementasi Flash Message

Flash Message digunakan untuk memberikan feedback kepada pengguna. Flash Message disimpan sementara menggunakan Session dan ditampilkan pada halaman tujuan setelah proses redirect.

3.9 Implementasi Post/Redirect/Get

Setelah berada pada halaman mahasiswa, pengguna dapat melakukan refresh tanpa menyebabkan form dikirim ulang. Dengan demikian pola PRG mampu mencegah terjadinya duplikasi data akibat refresh.

```

0 references | 0 overrides
153  ✓ public function store(): void
154  {
155      $result =
156          $this->service->create(
157              $_POST
158          );
159
160
161  ✓  if ($result['success']) {
162
163          $this->setFlash(
164              $result['message'],
165              'success'
166          );
167
168
169          $this->redirect(
170              '/mahasiswa'
171          );
172
173          return;
174      }
175

```

3.10 Implementasi CRUD Mahasiswa

CRUD mahasiswa yang berhasil diterapkan terdiri atas:

Create: Pengguna mengakses form tambah mahasiswa, mengisi data, kemudian menekan tombol Simpan. Data divalidasi oleh Service dan diteruskan ke Repository untuk disimpan ke MySQL.

Read: Data mahasiswa ditampilkan melalui halaman daftar mahasiswa. Repository mengambil data mahasiswa dan program studi dari database.

Update: Pengguna memilih menu Edit, mengubah data mahasiswa, kemudian Service melakukan validasi sebelum Repository menjalankan perintah UPDATE.

Delete: Pengguna menekan tombol Hapus. Permintaan diteruskan melalui Controller menuju Service dan selanjutnya Repository menjalankan DELETE menggunakan prepared statement.

3.11 Hasil Pengujian Data Valid

Pengujian dilakukan dengan mengisi seluruh data mahasiswa sesuai dengan ketentuan. Setelah tombol Simpan ditekan, data berhasil disimpan ke database dan aplikasi menampilkan Flash Message bahwa mahasiswa berhasil ditambahkan.

SI Akademik

Dashboard

Mahasiswa

Prodi

Mata Kuliah

Logout

Data mahasiswa berhasil ditambahkan.

Data Mahasiswa

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	261001	Ezra	ezra123@gmail.com	TK - Teknik Komputer	2026	Aktif	Detail Edit Hapus
2	251004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	SI - Sistem Informasi	2025	Aktif	Detail Edit Hapus
3	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	Detail Edit Hapus
4	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus
5	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

Pada pengujian ini nilai NIM dikosongkan. Service mendeteksi bahwa NIM tidak memenuhi validasi sehingga proses penyimpanan dihentikan

The screenshot shows a web browser window with the URL `localhost/Web%20Server/Acara%2014/si-akademik/public/mahasiswa/create`. The application header is 'SI Akademik' with navigation links: 'Dashboard', 'Mahasiswa', 'Prodi', 'Mata Kuliah', and 'Logout'. The main form is titled 'Tambah Data Mahasiswa' and contains the following fields:

- NIM: An empty text input field.
- Nama Mahasiswa: A text input field containing 'Nila Agustin Prima Aurelia'.
- Email: A text input field containing 'nilaaurelia97@gmail.com'.
- Program Studi: A dropdown menu showing 'MIK - Manajemen Informatika'.
- Angkatan: A text input field containing '2026'.

3.13 Pengujian NIM Duplikat

Pengujian dilakukan dengan menggunakan NIM yang telah tersimpan pada database. Method `existsByNim()` mendeteksi keberadaan NIM dan Service memberikan pesan

The screenshot shows the 'Edit Data Mahasiswa' form in the SI Akademik application. A red error message banner at the top states: 'NIM sudah digunakan oleh mahasiswa lain.' The form fields are:

- NIM: A text input field containing '261001'.
- Nama Mahasiswa: A text input field containing 'Ezra Faris'.
- Email: A text input field containing 'ezra123@gmail.com'.
- Program Studi: A dropdown menu showing 'TI - Teknik Informatika'.
- Angkatan: A text input field (empty).

The Windows taskbar at the bottom shows the date and time as 9:20 PM on 10/7/2026.

3.14 Pengujian Email Tidak Valid

Input email yang tidak memenuhi format email ditolak oleh validasi:

The screenshot shows a web form for editing student data. The form includes the following fields: Email (containing 'ezra123'), Program Studi (a dropdown menu with 'TI - Teknik Informatika' selected), Angkatan (containing '2026'), and Status Mahasiswa (a dropdown menu with 'Aktif' selected). A red error message box is displayed above the Program Studi dropdown, stating: 'Please include an '@' in the email address. 'ezra123' is missing an '@'. At the bottom right of the form, there are two buttons: 'Kembali' (Back) and 'Simpan Perubahan' (Save Changes).

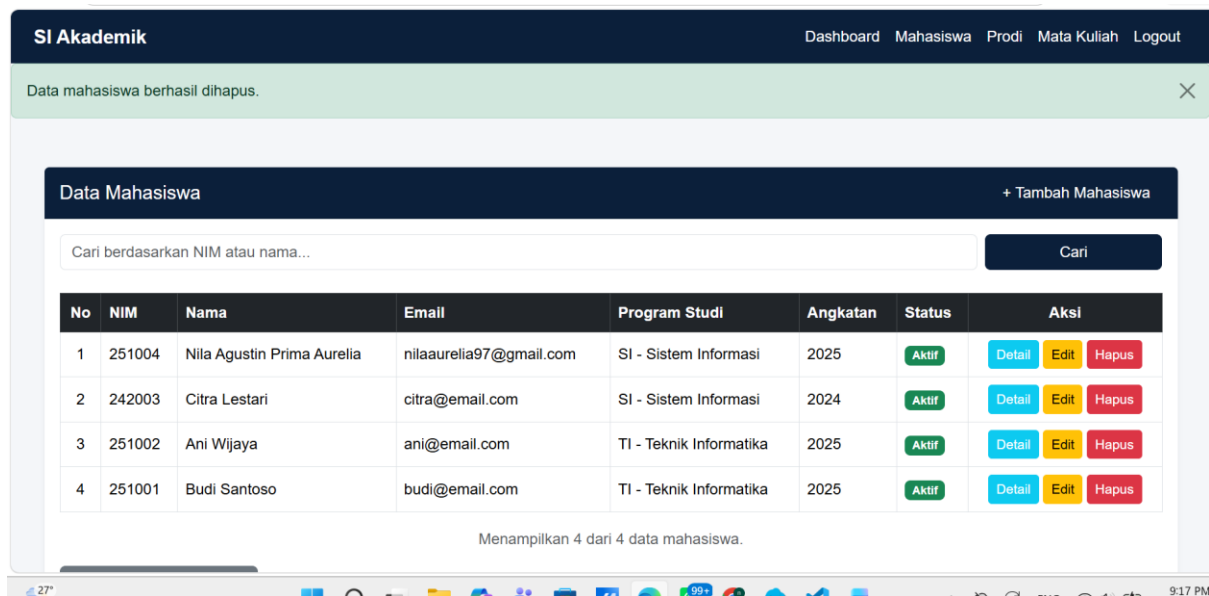
3.15 Hasil Pengujian Edit

Data mahasiswa dapat diubah melalui form Edit. Setelah data melewati validasi dan Repository menjalankan perintah UPDATE, aplikasi melakukan redirect menuju daftar mahasiswa dan menampilkan:



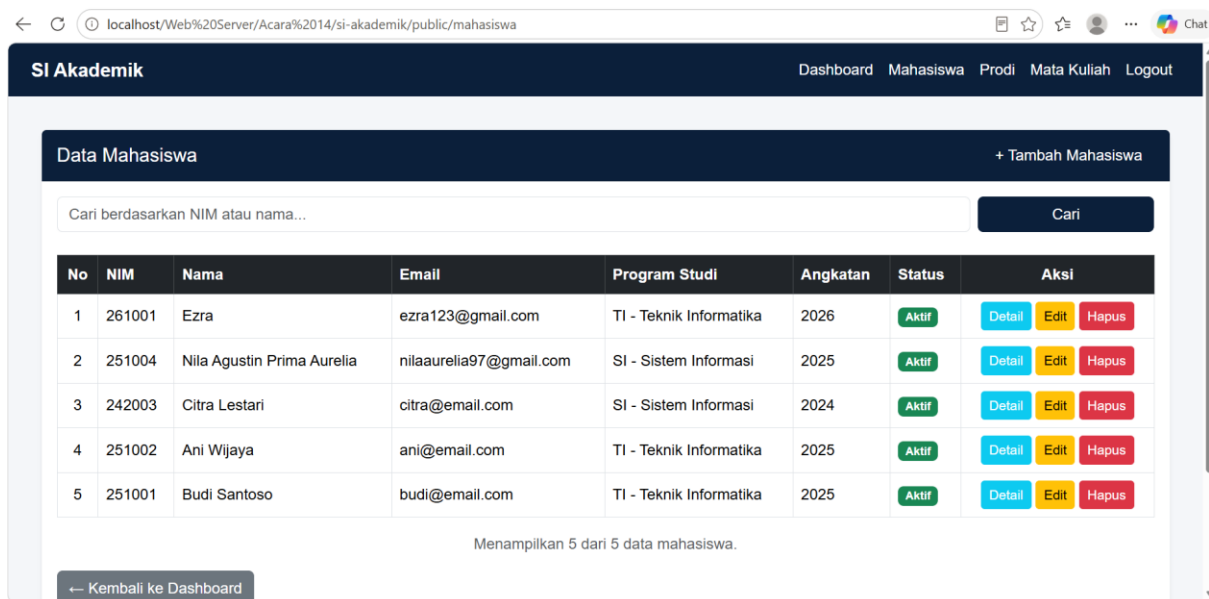
3.16 Hasil Pengujian Hapus

Mahasiswa dapat dihapus menggunakan tombol Hapus. Pengguna terlebih dahulu mendapatkan dialog konfirmasi. Setelah disetujui, proses berjalan melalui Controller, Service, Repository, kemudian MySQL.



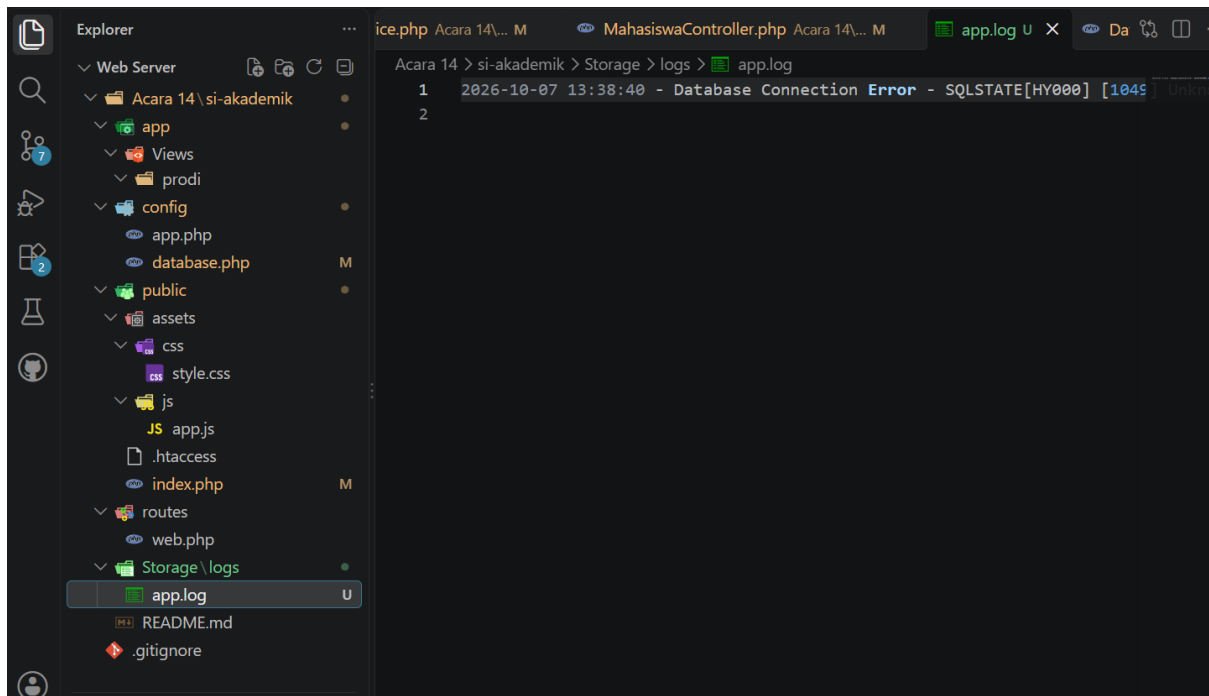
3.17 Hasil Pengujian PRG

Setelah menambahkan data mahasiswa, aplikasi melakukan redirect ke halaman daftar mahasiswa. Ketika halaman di-refresh, data mahasiswa tidak tersimpan kembali karena request terakhir brow ser adalah request GET.

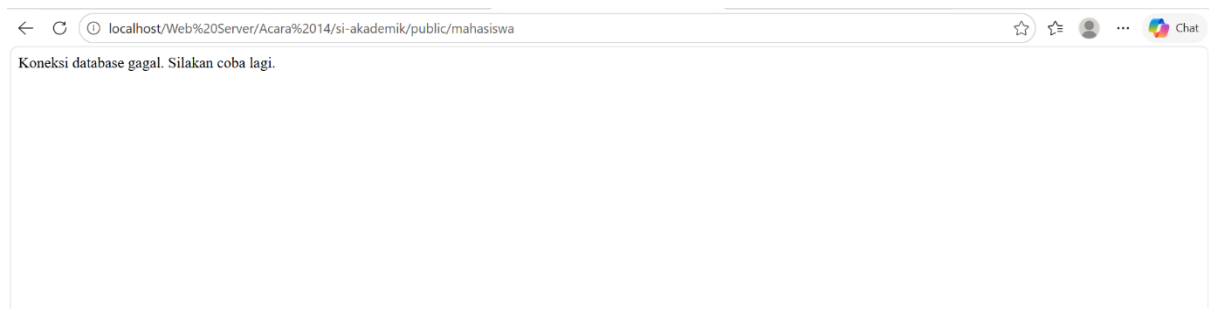


3.18 Hasil Pengujian Error Database dan Logging

Error database disimulasikan dengan menggunakan konfigurasi database yang tidak sesuai. Aplikasi tidak menampilkan informasi SQL kepada pengguna, tetapi hanya menampilkan pesan umum bahwa terjadi masalah pada database. Sementara itu, detail error berhasil disimpan ke:



Dengan demikian informasi teknis tetap tersedia untuk proses debugging oleh programmer tetapi tidak diberikan kepada pengguna.



Bab 4. Kesimpulan

Berdasarkan praktikum Acara 14 yang telah dilakukan, dapat disimpulkan bahwa aplikasi web yang baik tidak cukup hanya mampu menjalankan operasi CRUD, tetapi juga perlu memperhatikan keamanan, validasi data, penanganan error, logging, dan pengalaman pengguna.

Pada aplikasi Sistem Informasi Akademik, operasi CRUD mahasiswa telah terintegrasi dengan database MySQL menggunakan PDO. Seluruh query yang menerima input pengguna dijalankan menggunakan prepared statement sehingga data pengguna dipisahkan dari perintah SQL dan risiko SQL Injection dapat dikurangi.

Validasi data ditempatkan pada MahasiswaService sehingga data diperiksa sebelum diteruskan ke Repository. Service juga menangani business logic seperti pengecekan NIM duplikat serta mengatur proses Create, Update, dan Delete.

Exception Handling diterapkan menggunakan try-catch. Ketika terjadi kesalahan database, detail teknis tidak ditampilkan kepada pengguna. Detail kesalahan disimpan menggunakan Logging pada file storage/logs/app.log, sedangkan pengguna hanya menerima pesan yang sederhana dan aman melalui Flash Message.

Pola Post/Redirect/Get juga berhasil diterapkan setelah operasi yang menggunakan request POST. Setelah data berhasil diproses, aplikasi melakukan redirect menuju halaman daftar mahasiswa sehingga ketika pengguna melakukan refresh, form tidak dikirim ulang dan tidak terjadi duplikasi data.

Dengan demikian, praktikum ini menunjukkan bahwa alur aplikasi yang baik dapat dirancang sebagai User → Controller → Service → Validasi → Repository → MySQL. Apabila terjadi kesalahan, aplikasi dapat menerapkan alur Exception → Logging → Flash Message → Redirect. Penerapan tersebut membuat aplikasi lebih aman, terstruktur, mudah dipelihara, dan memberikan informasi yang lebih baik kepada pengguna.